

YORK UNIVERSITY
EECS 4413
Building e-Commerce Systems

Project Description

1. Introduction

For this project, you will implement an auction e-commerce system using open-source web-based frameworks. Your system will be composed of three major subsystems (i.e. tiers): a) the back-end subsystem which will contain all the modules implementing the business logic of the system and the corresponding databases; b) the middle subsystem which will use a framework (e.g. Spring Boot, Java EE) plus some application logic to relay incoming requests from the front-end subsystem (see next item) to the back-end and; c) the front-end subsystem which will use a framework for composing an HTML response back to the external client. You can implement the back-end subsystem in any language of your choice if the front-end subsystem uses standard REST service exposure and HTML with CSS (or any other standard styling framework) to relay responses back to the external clients, and the middle satisfies the modularity, microservice oriented requirements of specific milestones.

“An auction is usually a process of buying and selling goods or services by offering them up for bids, taking bids, and then selling the item to the highest bidder or buying the item from the lowest bidder”.
(Wikipedia).

For your project you will implement forward auctions:

- *Forward Auctions*, where a seller offers item(s) for sale and expects the highest price for the items they offer. These auctions are sales in which no fixed price is set for an item. Instead, an auctioneer receives bids on an item. Bids start from an initial price set by the auctioneer and each consequent bid must be strictly increasing; that is, if the last bid was \$25, the next one must be greater than \$25. Note bids are integer values. The auctioneer continues to receive bids from parties interested in the item until a fixed amount of time L has elapsed when the auction ends. Whomever makes the last highest bid wins and pays that amount for the item. In this type of auction, the bidders see the highest bidding price as well as the time remaining and decide to bid for a higher price or not. If no one bids for the item after the fixed amount of time L , then the item is removed from the list of auctioned items.

2. Use Cases

Below you find the minimum number of use cases. You can augment them with your own. Check also Milestone 3 for ideas about advanced use cases.

UC1: Sign-Up and Sign-In

In this use case the users are served with a page where they can either sign-up (first time users) or can sign-in (returning users). The initial screen provides the option to the user to either sign-up or sign-in by having a “Sign-Up” and a “Sign-In” button.

- **UC1.1 Sign-Up:** Signing-up requires the users to provide a username, a password, their first name, their last name and their shipping address (street name, street number, city, country, postal code). This information is then stored to a User-Database and will be used to verify consequent sign-ins. You can also consider two factor authentication.
- **UC1.2 Sign-In:** Signing-in requires the users to provide their username and passwords. If the combination of username and password is correct the user is allowed to sign in. This page should also have a “Forgot password” mechanism so the user can reset the password.

UC2: Browse Catalogue of Auctioned Items

- **UC2.1: Item Search.** Once logged in, the user provides a keyword and a list of items up for auction (or currently being auctioned) matching the keywords are displayed. You are free to have your own UI layout.
- **UC2.2: Display Auctioned Items.** The UI displays the full item name, the current bidding price, the type of auction (Forward), and the remaining time (applicable only for forward auctions i.e. highest bidder auctions)
- **UC2.3: Item selection.** Next to each displayed item there is a selection radio button which allows the user to select the item to bid for. The user can select only one item to bid for. Once the selection is made, the user can push the BID button and the UI displays the item details and the option to bid (see UC3 and Fig. 4). In this project we assume a bidder bids for only one item for each log-in session they have initiated. The user may have multiple log-in sessions in parallel in different browsers.

UC3: Bidding

In the case of Forward Auctions, the users can provide a new bidding price which must be higher than the current price and press the “BID” button (see Fig. 1). Once a bid is submitted, the page is refreshed (by the server), and the new highest bidding price and the highest bidder are *displayed to all users bidding for this item so that the current highest price and the ID of the current highest bidder are always displayed*. When the remaining time expires, the auction ends, and all users bidding for this item get the indication (i.e. a web page) that the auction has ended (see UC4 and Fig. 2). The users have the option to pay and are served a page with a “Pay Now” button (see UC4 below).

Fig. 1. Screen for Bidding on Forward Auctions

UC4: Auction Ended

Once the auction has ended the users are served a page “Pay Now” page with a “Pay Now” button. Only the user who won the item can pay, all others get a failure notice if they try to pay for an item for which they haven’t won. Note the option for expedited shipment that is added in this page. If the user selects this option, then the price of expedited shipment is added to the normal shipping price. This expedited shipment cost information can be included in the Catalogue DB for each specific item.

Fig. 2. The Auction Ended “Pay Now” page

UC5. Payment

Once the winner selects the “Pay Now” button, the winning bidder is served with a page to pay (Fig. 3). The page contains the user’s address (is served from the data stored in the database when the user signed-up). The user adds the payment details, that is the credit card number, the name on the card, the expiration date and the security code (**when testing, do not input real credit card info, just made-up names and numbers**). Once all added, the user can select the “Submit” button. All payment details have to be added so that the payment can commence. The payment includes the item price plus the shipping cost.

Fig. 3. Payment Page

UC 6. Receipt Page and Shipment Details

Once payment is cleared, the winning bidder is served with a receipt and shipment information. The page provides the total amount paid as well as the shipping details. You can consider that each item has a separate shipping time. You do not need to calculate the actual shipping date from the current date, just indicate that “The Item will be shipped in xxx days”. This information can be included in the Catalogue DB for each specific item.

Fig. 3. Receipt and Shipping Details Page

UC7. Auction item. A seller uploads information about an item to sell. The information contains a description of the item to sell, the type of auction (Forward) the duration of the auction, the starting bid price.

UC8-x. In Milestone 3 you will be asked to add a new advanced feature to the application, see that Milestone specifications. This feature is based on concepts learned in the class (make application self-adaptive, creates bids by conversing with AI bots etc.)

3. Design and Implementation Notes

Some suggestions on your design for the back-end include the following.

- You can have an Identity and Authorization Management (IAM) service (with a *User* class and most probably some more classes), along with the corresponding database which will manage users sign-up, sign-in, authentication credentials, authorizations, roles, etc.
- You can have a *Catalogue service* (with a *Catalogue* class – most probably some more classes too) along with a corresponding database which will serve all items which are up for auction and their time hasn't expired, or haven't been won yet. The database will also have information about the description of the item, the type of auction is offered, the shipping time, the initial price, and the end date of the auction (in the case of forward auctions (see Fig. 3)).
- You can have an *Auction* service (with an *Auction* class – most probably some more classes too) along with a corresponding database which will keep the bids for all items currently being auctioned, the highest price, the highest bidder, and the end time of the auction. Once an item is won then the database records that, and the item is also removed from the catalogue database.
- You can have a *Payment* service (with a *Payment* class – most probably some more classes too) along with a corresponding database which will keep the payment information for all items currently being auctioned, the highest price, the highest bidder, and the end time of the auction. Once an item is won, then the item is also removed from the catalogue database.
- You may want also to consider a *Gateway* service to be used as a Façade which orchestrates the whole process (e.g. bidding, pay-now, receipt) and keeps track of bidders, biddings, sessions, winners, etc. This will depend on your architecture design choices. All these services suggested above, can exchange information or call each other using any of the service communication protocols you have learnt in the class. The idea is on deliverable 3 you can distribute and deploy these as microservices in containers.
- Design the schema of your databases carefully so that the records in the different tables in the different data bases are linked with appropriate keys. It is suggested you use unique IDs as *CustomerIDs*, *ItemIDs*, and *BiddingIDs*. You can store the bidding history for an item as it evolves in the database of the *Auction* and/or the *Gateway*. Do not forget to include keywords for each item in the CatalogueDB so that search can be facilitated (see UC2).
- It is suggested you experiment with a lightweight relational database such as SQLite for your system and JDBC connectors if you use Java (highly suggested).
- Please make sure your architecture is very modular (i.e. each service runs as a separate process). You can achieve communication between all the services on the back-end using any protocol of your choice, Web sockets, XML-RPC, JSON-RPC, HTTP, etc, or pub/sub products such as Kafka or RabbitMQ.

4. Tools

You can use the following tools to proceed with the deliverables of your project:

A. Drawing UML Diagrams:

1. UMLet: UML design tool. Lightweight, pretty easy and intuitive to use. Download the standalone version “UMLet 15.0 stand-alone” from <https://www.umlet.com/changes.htm>
2. On-line UML drawing: <https://app.diagrams.net/>
3. MS-Visio (download the UML stencils) <https://support.microsoft.com/en-us/office/uml-diagrams-in-visio-ca4e3ae9-d413-4c94-8a7a-38dac30cbcd6?ui=en-us&rs=en-us&ad=us>

B. Automatically compiling class diagrams from your source code (useful for automatically acquiring an up-to-date visual overview of your code as a class diagram):

1. Object Aid: A very useful Eclipse plugin to automatically generate the class diagrams from your code. You can download it from <http://web.archive.org/web/20210111132410/https://www.objectaid.com/download>. Instructions on how to install it to Eclipse can be viewed at <https://www.youtube.com/watch?v=UtSRX9swc1w>

5. Deliverables

The project will have three Milestone/Deliverables.

- Milestone 1. The first deliverable focuses on system specification and architecture design issues. See Deliverable 1 instructions and software design document template provided to you for submitting Deliverable 1.
- Milestone 2. The Deliverable 2 will have the complete implementation of the back-end (i.e. business logic) services and all communications between these back-end services required to provide the full back-end system functionality. It is recommended, but not mandatory, that the back-end components are designed modularly so that they can be deployed as independent services in Milestone 3. The UI can be minimal, supporting individual use cases, without navigation. This milestone does not require user asynchronous notification.
- Milestone 3. The Deliverable 3 will have the full system functionality, including fully working UI, and the option to deploy the different back-end services as microservices on containers. This milestone also requires that the application includes the design and implementation of an *advanced feature using LLM* (see the specifications for Milestone 3)

6. Development and Runtime Frameworks

For Deliveries 2&3, you are allowed to use any technology presented in class and encouraged to go beyond the APIs covered in Labs, lectures or tutorials. **You can also use other technologies, not presented in the class. However, it is**

expected that each member of the team contributes to and understand each part of the project and therefore agrees on the technologies used in the project.

7. Team development and deployment

An ideal application, these days is composed of microservices, follows a DevOps process and is deployed on containers that run in clusters. Ideally, in Milestone 3, your application

- is deployed on containers, at a public address.
- on eclass you provide the design document, testcases, a link to the application.
- if you use cloud github, provide the link to it and give your TA access to it. Otherwise, provide your source files as jar/war file. The TA should see your code and scripts.

8. Acknowledgement of AI use.

In your design documents, you must explain how you use AI. You can use AI to increase your productivity. However, you are required to understand, explain and critique, change and maintain any artifact generated with the help of AI. Document the strength and weakness of GenAI as you experienced them.